NEXT ENGINEER · _SHEERAT_

Machine Learning

BOE073

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** ALL | **Semester:** 7 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 4th year, Open Elective-II, 2025-26 onwards

UNIT 1	UNIT 2	UNIT 3	UNIT 4	UNIT 5
Introduction and Concept Learning	Decision Trees and Artificial Neural Networks	Evaluating Hypotheses and Bayesian Learning	Computational Learning Theory and Instance-Based Learning	Genetic Algorithms, Rule Learning and Reinforcement Learning

Original revision notes written for the AKTU syllabus, which follows Tom Mitchell's "Machine Learning". Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: algorithms and comparison tables.
3. **Solved examples**: the numericals that appear every year (Find-S, candidate elimination, ID3 entropy and information gain, perceptron training, confidence intervals, Naive Bayes, PAC sample complexity, k-NN, GA crossover and selection, Q-learning).
4. **Formulas / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (learning problems, designing a learning system, concept learning, Find-S, candidate elimination, inductive bias) and Unit 2 (ID3, entropy, information gain, overfitting, perceptron, delta rule, Adaline, backpropagation).
- **Afternoon (3 hours):** Unit 3 (sample error, true error, confidence intervals, comparing algorithms, Bayes theorem, MAP, Bayes optimal classifier, Naive Bayes, belief networks, EM) and Unit 4 (PAC learning, sample complexity, VC dimension, mistake bound, k-NN, locally weighted regression, RBF, case-based reasoning).
- **Evening (2 hours):** Unit 5 (genetic algorithms, genetic programming, Baldwin effect, sequential covering, FOIL, reinforcement learning, Q-learning). Then the "Last-hour checklist".
- **In the exam:** for ID3 show every entropy and gain value; for Naive Bayes write each probability before multiplying; for candidate elimination show S and G after every example.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions

(one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

NEXT ENGINEER
sheerat

UNIT 1 Introduction and Concept Learning

✍️ 2-mark definitions

- **Machine learning (Mitchell):** a computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.
- **Well-posed learning problem:** one where T, P and E are clearly specified. Example: checkers: T = playing checkers, P = percentage of games won, E = games played against itself.
- **Target function:** the ideal function the learner tries to approximate, for example V: Board → ℝ giving the value of a board state.
- **Concept learning:** inferring a boolean-valued function (a concept) from training examples of its input and output.
- **Hypothesis:** a candidate description of the target concept; in concept learning a conjunction of attribute constraints such as ⟨Sunny, ?, ?, Strong, ?, ?⟩.
- **Hypothesis space H:** the set of all hypotheses the learner may consider.
- **Consistent hypothesis:** one that classifies every training example correctly, h(x) = c(x).
- **More-general-than-or-equal (≥g):** h_j ≥g h_k if every instance that satisfies h_k also satisfies h_j.
- **Version space:** the subset of H consistent with all training examples, represented by its general boundary G and specific boundary S.
- **Inductive bias:** the set of assumptions a learner uses to predict outputs for inputs it has not seen; without bias a learner cannot generalize.

💡 Core concepts

Designing a learning system (checkers example):

1. **Choose the training experience:** direct or indirect feedback, how much the learner controls the examples, how well the training distribution matches the test distribution.
2. **Choose the target function:** V(b) giving the board's value (100 win, −100 loss, 0 draw).
3. **Choose a representation:** $\hat{V}(b) = w0 + w1 \times 1 + \ldots + w6 \times 6$ where x_i are board features (number of pieces, kings, threatened pieces).
4. **Choose a learning algorithm:** estimate training values $V_{train}(b) \leftarrow \hat{V}(Successor(b))$ and adjust weights with LMS: $w_i \leftarrow w_i + \eta\,(V_{train}(b) - \hat{V}(b))\,x_i$.
5. **Final design:** four modules: Performance System, Critic, Generalizer, Experiment Generator.

Issues in machine learning: which algorithms work for which problems; how much data is enough; how prior knowledge helps; choosing training experiences; reducing learning to function approximation; changing representation automatically.

Types of learning: supervised (labelled data: classification, regression), unsupervised (clustering), reinforcement (rewards), semi-supervised.

Find-S algorithm (finds the maximally specific hypothesis)

1. Start with h = ⟨ø, ø, ..., ø⟩ (most specific).
2. For each positive example x: for each attribute, if the constraint is satisfied do nothing; otherwise replace it with the next more general constraint (value, then ?).
3. Ignore negative examples. Output h.

Limitations: ignores negatives, cannot tell whether it found the only consistent hypothesis, fails with noisy data, may have several maximally specific hypotheses.

List-Then-Eliminate: list every hypothesis in H, remove any inconsistent with each example; outputs the version space. Works only for tiny finite H.

Candidate elimination algorithm

- Initialize G to the most general hypothesis ⟨?, ?, ..., ?⟩ and S to the most specific ⟨ø, ..., ø⟩.
- For a **positive** example: remove from G any hypothesis inconsistent with it; minimally generalize S to cover it (keeping only hypotheses more specific than some g in G).
- For a **negative** example: remove from S any hypothesis that covers it; minimally specialize G so it excludes the example (keeping only hypotheses more general than some s in S).
- The version space is everything between S and G.

Inductive bias: candidate elimination assumes the target concept is in H (conjunctions of attribute values). An **unbiased learner** (H = power set of instances) can only memorize, never generalize. Ranking of bias strength: rote learner (no bias) < candidate elimination (target in H) < Find-S (target in H and all instances negative unless proved positive).

🧮 Solved examples

EnjoySport training data

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Example 1: Find-S.

- h0 = ⟨ø, ø, ø, ø, ø, ø⟩.
- After 1: ⟨Sunny, Warm, Normal, Strong, Warm, Same⟩.
- After 2: ⟨Sunny, Warm, ?, Strong, Warm, Same⟩.
- Example 3 is negative, ignored.
- After 4: **⟨Sunny, Warm, ?, Strong, ?, ?⟩**.

Example 2: Candidate elimination.

- S0 = ⟨ø, ø, ø, ø, ø, ø⟩; G0 = ⟨?, ?, ?, ?, ?, ?⟩.
- After 1 (+): S1 = ⟨Sunny, Warm, Normal, Strong, Warm, Same⟩; G1 = G0.
- After 2 (+): S2 = ⟨Sunny, Warm, ?, Strong, Warm, Same⟩; G2 = G0.
- After 3 (−): S3 = S2; G3 = {⟨Sunny, ?, ?, ?, ?, ?⟩, ⟨?, Warm, ?, ?, ?, ?⟩, ⟨?, ?, ?, ?, ?, Same⟩}.

- After 4 (+): S4 = ⟨Sunny, Warm, ?, Strong, ?, ?⟩; ⟨?, ?, ?, ?, ?, Same⟩ is removed from G because it fails example 4. G4 = {⟨Sunny, ?, ?, ?, ?, ?⟩, ⟨?, Warm, ?, ?, ?, ?⟩}.
- The version space has six hypotheses between S4 and G4 (S4 itself, the two G4 members, and ⟨Sunny, ?, ?, Strong, ?, ?⟩, ⟨Sunny, Warm, ?, ?, ?, ?⟩, ⟨?, Warm, ?, Strong, ?, ?⟩).

Example 3: Size of the hypothesis space. Sky has 3 values and the other five attributes have 2 each. Instances = 3·2·2·2·2·2 = 96. Syntactically distinct hypotheses (each attribute can also be ? or ∅) = 5·4·4·4·4·4 = 5120. Semantically distinct (all ∅ hypotheses are the same) = 1 + 4·3·3·3·3·3 = 973.

Key points to remember

- Mitchell's definition with T, P, E.
- Design steps: experience, target function, representation, algorithm (LMS).
- Find-S uses positives only; candidate elimination keeps S and G.
- Inductive bias is essential; an unbiased learner cannot generalize.

⭐ Important questions

1. Define machine learning and a well-posed learning problem with examples.
2. Explain the steps in designing a learning system with the checkers example.
3. Explain the issues in machine learning.
4. Explain concept learning as search and the general-to-specific ordering. Explain Find-S with an example.
5. Explain the candidate elimination algorithm with an example. What is a version space?
6. What is inductive bias? Why is it needed?

UNIT 2 Decision Trees and Artificial Neural Networks

✍️ 2-mark definitions

- **Decision tree:** a tree in which internal nodes test attributes, branches are attribute values and leaves give the classification.
- **Entropy:** the impurity of a set of examples: Entropy(S) = $-p_+ \log_2 p_+ - p_- \log_2 p_-$.
- **Information gain:** the expected reduction in entropy from splitting on attribute A: Gain(S, A) = Entropy(S) − Σ ($|S_v| / |S|$) Entropy(S_v).
- **Overfitting:** a hypothesis that fits the training data very well but performs worse on unseen data than a simpler hypothesis would.
- **Pruning:** removing parts of a tree that do not improve accuracy on validation data, to reduce overfitting.
- **Perceptron:** a single neuron that outputs 1 if $w \cdot x + b > 0$ and −1 (or 0) otherwise.
- **Delta rule (LMS, Widrow-Hoff):** gradient descent on squared error for a linear unit: $\Delta w_i = \eta \Sigma (t - o) x_i$.
- **Adaline:** Adaptive Linear Neuron, a linear unit trained with the delta rule on the net input before thresholding.
- **Backpropagation:** a gradient descent algorithm for multilayer networks that propagates error terms backward from the output layer to compute weight updates.
- **Sigmoid function:** σ(y) = 1 / (1 + e^(−y)); its derivative is σ(y)(1 − σ(y)).

💡 Core concepts

Appropriate problems for decision trees: instances described by attribute-value pairs, discrete outputs, disjunctive descriptions needed, noisy training data, missing values.

ID3 algorithm: at each node choose the attribute with the highest information gain, create a branch for each value, partition the examples, and recurse until all examples at a node have the same class or no attributes remain. Inductive bias of ID3: prefers shorter trees with high-gain attributes near the root (Occam's razor).

Issues in decision tree learning

Issue	Solution
Overfitting	Stop growing early, or reduced-error pruning, rule post-pruning (C4.5)
Continuous attributes	Choose a threshold that maximizes gain
Attributes with many values (e.g. Date)	Gain ratio = Gain / SplitInformation
Missing values	Use most common value or fractional examples
Attributes with costs	$Gain^2$ / Cost, or similar measures

Perceptron training rule: $w_i \leftarrow w_i + \eta (t - o) x_i$. It converges if the data are linearly separable and η is small enough. A single perceptron can represent AND, OR, NAND, NOR but not XOR.

Gradient descent and the delta rule: error $E(w) = \frac{1}{2} \Sigma_d (t_d - o_d)^2$, where $o = w \cdot x$ (unthresholded). $\Delta w_i = \eta \Sigma_d (t_d - o_d) x_{id}$. Converges to the minimum-error weights even if data are not separable. Stochastic (incremental) version updates after each example.

Perceptron rule vs delta rule

Perceptron rule	Delta rule
Uses thresholded output	Uses linear (unthresholded) output
Converges in finite steps if separable	Converges asymptotically to least squared error
May not converge if not separable	Converges even if not separable

Multilayer networks and backpropagation (sigmoid units)

1. Forward pass: compute outputs of every unit.
2. Output unit error: $\delta_k = o_k (1 - o_k)(t_k - o_k)$.
3. Hidden unit error: $\delta_h = o_h (1 - o_h) \Sigma_k w_{kh} \delta_k$.
4. Update: $w_{ji} \leftarrow w_{ji} + \eta \delta_j x_{ji}$ (plus momentum $\alpha \Delta w_{ji}(n - 1)$ if used).

Convergence and local minima: backprop finds a local minimum; remedies: momentum, stochastic updates, multiple random restarts. Representational power: any boolean function with 2 layers; any bounded continuous function with one hidden layer; any function with two hidden layers. **Generalization:** stop training when validation error starts to rise (early stopping), weight decay.

🧮 Solved examples

Example 1: ID3 on PlayTennis (14 examples, 9 Yes, 5 No).

- Entropy(S) = $-(9/14) \log_2(9/14) - (5/14) \log_2(5/14)$ = **0.940**.
- Outlook: Sunny [2+, 3−] entropy 0.971; Overcast [4+, 0−] entropy 0; Rain [3+, 2−] entropy 0.971. Gain = $0.940 - (5/14)(0.971) - 0 - (5/14)(0.971)$ = **0.247**.
- Humidity: High [3+, 4−] 0.985; Normal [6+, 1−] 0.592. Gain = **0.152**.
- Wind: Weak [6+, 2−] 0.811; Strong [3+, 3−] 1.0. Gain = **0.048**.
- Temperature: Hot [2+, 2−], Mild [4+, 2−], Cool [3+, 1−]. Gain = **0.029**.
- **Root = Outlook**. Overcast branch is a Yes leaf.
- Sunny branch (5 examples, entropy 0.971): Gain(Humidity) = 0.971 (pure split), Gain(Temperature) = 0.571, Gain(Wind) = 0.020, so split on **Humidity** (High → No, Normal → Yes). Rain branch splits on **Wind** (Weak → Yes, Strong → No).

Example 2: Perceptron learning OR. Start $w = (0, 0)$, $b = 0$, $\eta = 1$, output 1 if $w \cdot x + b > 0$ else 0. Examples in order (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→1.

- Epoch 1: (0,0) net 0, out 0, correct. (0,1) net 0, out 0, target 1: $w = (0, 1)$, $b = 1$. (1,0) net 1, correct. (1,1) net 2, correct.
- Epoch 2: (0,0) net 1, out 1, target 0: $b = 0$. (0,1) net 1, correct. (1,0) net 0, out 0, target 1: $w = (1, 1)$, $b = 1$. (1,1) correct.

- Epoch 3: (0,0) net 1, wrong: b = 0. Others correct.
- Epoch 4: all correct. **Final w = (1, 1), b = 0.**

Example 3: One backprop step. One sigmoid output unit, input x = 1 (and bias input 1), weights w = 0.5, w0 = 0, target t = 1, η = 0.5. net = 0.5, o = σ(0.5) = 0.6225. δ = o(1 − o)(t − o) = 0.6225 × 0.3775 × 0.3775 = 0.0887. Δw = 0.5 × 0.0887 × 1 = 0.0444, so w = 0.5444 and w0 = 0.0444.

Formulas

Item	Formula
Entropy	$-\Sigma\ p_i \log_2 p_i$
Gain	$\text{Entropy}(S) - \Sigma\ (\lvert S_v\rvert/\lvert S\rvert)\ \text{Entropy}(S_v)$
Gain ratio	$\text{Gain} / (-\Sigma\ (\lvert S_i\rvert/\lvert S\rvert) \log_2(\lvert S_i\rvert/\lvert S\rvert))$
Perceptron rule	$w_i \leftarrow w_i + \eta(t - o)x_i$
Delta rule	$\Delta w_i = \eta\ \Sigma(t - o)x_i$
Output δ	$o(1 - o)(t - o)$
Hidden δ	$o_h(1 - o_h)\ \Sigma\ w_{kh}\ \delta_k$

⭐ Important questions

1. Explain the ID3 algorithm. Construct a decision tree for a given dataset using information gain.
2. Explain the issues in decision tree learning: overfitting, continuous attributes, missing values, gain ratio.
3. Explain the perceptron and its training rule. Why can it not learn XOR?
4. Explain gradient descent and the delta rule. Compare it with the perceptron rule. What is Adaline?
5. Derive and explain the backpropagation algorithm. Discuss convergence, local minima and generalization.

UNIT 3 Evaluating Hypotheses and Bayesian Learning

✍️ 2-mark definitions

- **Sample error:** the fraction of a sample S that h misclassifies: error_S(h) = r / n.
- **True error:** the probability that h misclassifies an instance drawn at random from distribution D.
- **Confidence interval:** a range that contains the true error with a stated probability, for example 95%.
- **Bias of an estimator:** E[estimate] − true value; sample error on training data is optimistically biased, so use independent test data.
- **Bayes theorem:** P(h | D) = P(D | h) P(h) / P(D).
- **MAP hypothesis:** the hypothesis with the highest posterior probability: h_MAP = argmax P(D | h) P(h).
- **ML hypothesis:** the hypothesis that maximizes P(D | h) (MAP with equal priors).
- **Bayes optimal classifier:** classifies by weighting every hypothesis's prediction by its posterior: argmax over v of Σ_h P(v | h) P(h | D). No other method using the same H and prior does better on average.
- **Naive Bayes classifier:** assumes attributes are conditionally independent given the class: v_NB = argmax P(v) Π P(a_i | v).
- **Bayesian belief network:** a directed acyclic graph whose nodes are variables, with conditional probability tables, encoding conditional independence.
- **EM algorithm:** Expectation-Maximization, an iterative method to find ML estimates when some variables are hidden: the E-step estimates hidden values, the M-step re-estimates parameters.

💡 Core concepts

Estimating hypothesis accuracy: with n ≥ 30 test examples, the N% confidence interval for true error is error_S(h) ± z_N √(error_S(h)(1 − error_S(h)) / n). z values: 90% → 1.64, 95% → 1.96, 99% → 2.58.

Basics of sampling theory: the number of errors r follows a binomial distribution; mean nP, variance nP(1 − P); for large n it is close to normal. Standard deviation of error_S ≈ √(e(1 − e)/n).

Difference in error of two hypotheses: d̂ = error_S1(h1) − error_S2(h2), σ ≈ √(e1(1 − e1)/n1 + e2(1 − e2)/n2); interval d̂ ± z_N σ.

Comparing learning algorithms: k-fold cross-validation: split data into k parts, for each part train both algorithms on the rest and test on it, compute the mean difference $\bar{\delta}$ and its interval with the t-distribution (paired t-test): $\bar{\delta} \pm t_{N, k-1}\, s_{\bar{\delta}}$.

Bayes theorem and concept learning: with a uniform prior and noise-free data, every consistent hypothesis is a MAP hypothesis, P(h | D) = 1/|VS|. Minimizing squared error gives the ML hypothesis under Gaussian noise. Minimum Description Length principle: prefer h minimizing L(h) + L(D | h).

Naive Bayes: estimate P(v_j) and P(a_i | v_j) from frequencies; use the m-estimate (n_c + m p) / (n + m) to avoid zero probabilities. Widely used for text classification.

Bayesian belief networks: each node is conditionally independent of its non-descendants given its parents. Joint probability P(y1, ..., yn) = Π P(y_i | Parents(Y_i)). Inference is exact for small networks and approximate (sampling) for large ones; learning uses gradient ascent when some variables are hidden.

EM algorithm (mixture of k Gaussians example): start with random means. E-step: compute the expected value of each hidden variable z_ij (probability that x_i came from Gaussian j). M-step: update each mean μ_j = Σ E[z_ij] x_i / Σ E[z_ij]. Repeat until convergence; it finds a local maximum of the likelihood. k-means is a hard-assignment special case.

🧮 Solved examples

Example 1: Confidence interval. A hypothesis makes 12 errors on 40 test examples. error_S = 0.30. 95% interval = 0.30 ± 1.96 √(0.30 × 0.70 / 40) = 0.30 ± 1.96 × 0.0725 = 0.30 ± 0.142, so **0.158 to 0.442**.

Example 2: Naive Bayes on PlayTennis. Classify ⟨Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong⟩.

- P(Yes) = 9/14, P(No) = 5/14.
- P(Sunny | Yes) = 2/9, P(Cool | Yes) = 3/9, P(High | Yes) = 3/9, P(Strong | Yes) = 3/9.
- P(Sunny | No) = 3/5, P(Cool | No) = 1/5, P(High | No) = 4/5, P(Strong | No) = 3/5.
- Yes: 9/14 × 2/9 × 3/9 × 3/9 × 3/9 = **0.0053**.
- No: 5/14 × 3/5 × 1/5 × 4/5 × 3/5 = **0.0206**.
- Classify as **No**; normalized P(No) = 0.0206 / (0.0206 + 0.0053) = 0.795.

Example 3: Bayes theorem (medical test). Cancer prior 0.008; test positive for 98% of cancer cases and for 3% of healthy people. P(+ | cancer) P(cancer) = 0.98 × 0.008 = 0.0078; P(+ | ¬cancer) P(¬cancer) = 0.03 × 0.992 = 0.0298. h_MAP = ¬cancer. P(cancer | +) = 0.0078 / (0.0078 + 0.0298) = **0.21**.

Example 4: Bayes optimal vs MAP. Three hypotheses with posteriors 0.4, 0.3, 0.3. h1 predicts +, h2 and h3 predict −. MAP hypothesis h1 says +, but the Bayes optimal classifier sums: P(+) = 0.4, P(−) = 0.6, so it says **−**.

Key points to remember

- CI: e ± z √(e(1 − e)/n); z95 = 1.96.
- Bayes: posterior ∝ likelihood × prior; MAP vs ML.
- Bayes optimal combines all hypotheses; Naive Bayes assumes conditional independence.
- Belief network joint = product of P(node | parents).
- EM: E-step expectations, M-step maximize; local optimum.

⭐ Important questions

1. Explain how to estimate hypothesis accuracy. Derive confidence intervals for sample error.
2. Explain the basics of sampling theory and comparing two learning algorithms.
3. Explain Bayes theorem, MAP and ML hypotheses and their relation to concept learning.

4. Explain the Bayes optimal classifier and the Naive Bayes classifier with an example.
5. Explain Bayesian belief networks.
6. Explain the EM algorithm.

NEXT ENGINEER
sheerat

UNIT 4 Computational Learning Theory and Instance-Based Learning

✍️ 2-mark definitions

- **Computational learning theory (COLT):** the study of how many examples, how much computation and how many mistakes are needed to learn a concept.
- **PAC learning:** Probably Approximately Correct: a class C is PAC-learnable if, for any ε and δ, the learner outputs with probability at least (1 − δ) a hypothesis with true error at most ε, in time polynomial in 1/ε, 1/δ, n and size(c).
- **Sample complexity:** the number of training examples needed to learn successfully.
- **ε-exhausted version space:** every hypothesis left in the version space has true error less than ε.
- **VC dimension:** the size of the largest set of instances that H can shatter (label in every possible way).
- **Shattering:** H shatters a set S if for every dichotomy of S some hypothesis in H is consistent with it.
- **Mistake bound model:** measures the total number of mistakes a learner makes before it converges to the correct hypothesis.
- **Instance-based (lazy) learning:** storing training examples and deferring generalization until a query arrives (k-NN, locally weighted regression, case-based reasoning).
- **k-nearest neighbour:** classifies a query by the majority label of its k closest training examples (Euclidean distance).
- **Radial basis function (RBF) network:** a network whose hidden units compute Gaussian kernels centred at points, $f(x) = w0 + \Sigma\ w_u\ K_u(d(x_u, x))$.
- **Case-based reasoning (CBR):** solving new problems by retrieving and adapting solutions of similar past cases represented symbolically.

💡 Core concepts

Sample complexity for finite H (consistent learner): $m \geq (1/\varepsilon)(\ln|H| + \ln(1/\delta))$. For agnostic learning (target may not be in H): $m \geq (1/(2\varepsilon^2))(\ln|H| + \ln(1/\delta))$.

Infinite H: $m \geq (1/\varepsilon)(4 \log_2(2/\delta) + 8\ VC(H) \log_2(13/\varepsilon))$. VC dimension examples: intervals on the real line = 2; linear separators in the plane = 3; linear separators in n dimensions = n + 1. $VC(H) \leq \log_2|H|$.

Mistake bound model: Find-S for conjunctions of n boolean literals makes at most n + 1 mistakes. The Halving algorithm (majority vote of the version space) makes at most $\log_2|H|$ mistakes. Weighted-Majority makes at most $2.4(k + \log_2 n)$ mistakes, where k is the mistakes of the best of n predictors. Optimal mistake bound: $VC(C) \leq Opt(C) \leq \log_2|C|$.

k-NN details: for real-valued targets use the mean of the k neighbours. **Distance-weighted k-NN:** weight $w_i = 1/d(x_q, x_i)^2$. Problems: curse of dimensionality (irrelevant attributes dominate distance), so scale or weight attributes; slow queries, so use kd-trees. Inductive bias: the class of a point is most like nearby points.

Locally weighted regression: fit a local model (linear) around each query, weighting examples by a kernel of distance; minimize $E = \tfrac{1}{2} \Sigma (f(x) - \hat{f}(x))^2 K(d(x_q, x))$ over the k nearest neighbours.

RBF networks: training in two stages: choose centres (all examples, a subset, or EM/k-means clusters) and widths; then train the output weights (linear) by least squares. Fast to train; local approximation.

Case-based reasoning (CADET example): cases are rich symbolic descriptions (function graphs of plumbing devices); retrieve similar cases, combine and adapt them, possibly with domain knowledge. Steps: retrieve, reuse, revise, retain.

Lazy vs eager learning

Lazy (k-NN, CBR)	Eager (decision tree, backprop)
Generalizes at query time	Generalizes at training time
Fast training, slow query	Slow training, fast query
Local approximation per query	One global approximation
Stores all data	Stores a compact model

🧮 Solved examples

Example 1: PAC sample complexity. H = conjunctions of up to 10 boolean literals, so $|H| = 3^{10}$. With $\varepsilon = 0.1$ and $\delta = 0.05$: $m \geq (1/0.1)(10 \ln 3 + \ln 20) = 10 \times (10.99 + 3.00) = 139.8$, so **140 examples**.

Example 2: k-NN. Training points: (1,1) A, (2,1) A, (4,4) B, (5,4) B, (4,5) B. Query (2,2), k = 3.

- Distances: (2,1) 1.0; (1,1) 1.41; (4,4) 2.83; (5,4) 3.61; (4,5) 3.61.
- Three nearest: A, A, B, so the class is **A**.
- With k = 5 the vote is 2 A to 3 B, so B; with distance weighting $(1/d^2)$: A = 1 + 0.5 = 1.5, B = 0.125 + 0.077 + 0.077 = 0.279, so **A**. This shows why the choice of k and weighting matters.

Example 3: VC dimension of intervals. Two points $x1 < x2$ can be labelled in all 4 ways by intervals, so VC ≥ 2. Three points $x1 < x2 < x3$ labelled +, −, + cannot be covered by one interval without including x2, so VC = 2.

Formulas

Item	Formula
Finite H, consistent	$m \geq (1/\varepsilon)(\ln\|H\| + \ln(1/\delta))$
Agnostic	$m \geq (1/2\varepsilon^2)(\ln\|H\| + \ln(1/\delta))$
VC bound	$m \geq (1/\varepsilon)(4 \log_2(2/\delta) + 8\, VC(H) \log_2(13/\varepsilon))$
Halving mistakes	$\leq \log_2\|H\|$
Distance weight	$w_i = 1 / d(x_q, x_i)^2$
RBF	$f(x) = w0 + \Sigma\, w_u \exp(-d^2(x_u, x) / 2\sigma_u^2)$

⭐ Important questions

1. What is PAC learning? Derive the sample complexity for finite hypothesis spaces.
2. Explain sample complexity for infinite hypothesis spaces and the VC dimension.
3. Explain the mistake bound model of learning (Find-S, Halving, Weighted-Majority).
4. Explain the k-nearest neighbour algorithm with an example. What is distance-weighted k-NN?
5. Explain locally weighted regression and RBF networks.
6. Explain case-based reasoning. Compare lazy and eager learning.

NEXT ENGINEER
sheerat

UNIT 5 Genetic Algorithms, Rule Learning and Reinforcement Learning

✍️ 2-mark definitions

- **Genetic algorithm (GA):** a search method inspired by natural evolution that keeps a population of hypotheses (bit strings) and improves it by selection, crossover and mutation according to a fitness function.
- **Fitness function:** the measure of how good a hypothesis is (for example accuracy on training data).
- **Crossover:** creating offspring by combining parts of two parent strings (single-point, two-point, uniform).
- **Mutation:** randomly flipping a bit in a string to keep diversity.
- **Genetic programming (GP):** evolving computer programs represented as parse trees, with crossover swapping sub-trees.
- **Baldwin effect:** individual learning (within a lifetime) can speed up evolution, because individuals that can learn survive and the population later evolves the learned traits.
- **Lamarckian evolution:** the (biologically false) idea that traits learned in a lifetime pass directly to offspring; it can work in computer GAs.
- **Sequential covering:** learning a set of rules one at a time, each covering some positive examples, then removing those examples and repeating.
- **FOIL:** First-Order Inductive Learner, which learns Horn clause (first-order) rules by general-to-specific search using FOIL-Gain.
- **Reinforcement learning (RL):** learning to act by trial and error to maximize cumulative reward from the environment.
- **Q-learning:** a model-free RL algorithm that learns Q(s, a), the value of taking action a in state s and acting optimally afterwards.
- **Discount factor γ:** the weight ($0 \le \gamma < 1$) on future rewards; $V = r_t + \gamma\, r_{(t+1)} + \gamma^2\, r_{(t+2)} + \ldots$

💡 Core concepts

Genetic algorithm steps

1. Initialize a population of p random hypotheses.
2. Evaluate fitness of each.
3. Select parents with probability proportional to fitness (roulette wheel): $Pr(h_i) = Fitness(h_i) / \Sigma\, Fitness(h_j)$. Other methods: tournament, rank selection.
4. Apply crossover to a fraction r of the population and mutation with rate m.
5. Replace the population with the new generation; repeat until fitness reaches a threshold.

Representing hypotheses: rules encoded as bit strings, for example Outlook (Sunny, Overcast, Rain) as 3 bits, 011 meaning Overcast or Rain. **GABIL** learns rule sets of variable length.

Hypothesis space search: GA search is randomized and parallel; it can jump in the space and avoid some local minima. **Crowding** (similar individuals take over) is reduced by fitness sharing and restricted mating.

Models of evolution and learning: Lamarckian (learned traits inherited directly) and the Baldwin effect (learning guides evolution indirectly).

Genetic programming: programs as trees of functions (+, sin, √) and terminals (x, constants); crossover swaps subtrees; used to evolve formulas and controllers (block-stacking example in Mitchell).

Sequential covering algorithm

- Learn-One-Rule (general-to-specific beam search) finds a rule with high accuracy; remove the positive examples it covers; repeat until no more good rules; sort rules by performance.
- Variations: CN2, AQ, RIPPER. Simultaneous covering (decision trees) vs sequential covering (rule sets).

Learning first-order rules (FOIL)

- Learns rules like GrandDaughter(x, y) ← Father(y, z) ∧ Father(z, x) ∧ Female(y).
- Outer loop: sequential covering of positive examples. Inner loop: add literals to the rule body one at a time, choosing the literal with the best FOIL_Gain(L, R) = t ($\log_2$(p1/(p1 + n1)) − $\log_2$(p0/(p0 + n0))), where p0, n0 are positive and negative bindings before adding L, p1, n1 after, and t is the number of positive bindings still covered.
- Can learn recursive rules (Ancestor).

Reinforcement learning

- Agent observes state s, takes action a, gets reward r and new state s'. Goal: a policy π maximizing discounted return.
- Differs from supervised learning: delayed reward, exploration vs exploitation, partially observable states, life-long learning.
- **Q-learning (deterministic):** Q̂(s, a) ← r + γ max_a' Q̂(s', a'). **Non-deterministic:** $\hat{Q}_n(s, a) \leftarrow (1 - \alpha_n)\,\hat{Q}_{(n-1)}(s, a) + \alpha_n [r + \gamma \max_{a'} \hat{Q}_{(n-1)}(s', a')]$. The optimal policy is π*(s) = argmax_a Q(s, a).
- Converges if every state-action pair is visited infinitely often and rewards are bounded.
- Exploration: choose actions with probability ∝ k^Q̂(s,a) (or ε-greedy).
- **Temporal difference learning TD(λ)** blends one-step and multi-step lookahead.

🧮 Solved examples

Example 1: Roulette wheel selection. Fitness $f(x) = x^2$ for population x = 13, 24, 8, 19: fitness 169, 576, 64, 361 (total 1170). Selection probabilities: 0.144, 0.492, 0.055, 0.309. The string for 24 is most likely to be chosen.

Example 2: Crossover and mutation. Parents 11001|010 and 00101|111 with single-point crossover after bit 5 give offspring **11001111** and **00101010**. Mutating bit 2 of the first offspring gives 10001111.

Example 3: Q-learning update (deterministic). γ = 0.9. The agent in state s1 moves right to s2 with r = 0. Current Q̂(s2, ·) values are 63, 81 and 100. Update Q̂(s1, right) = 0 + 0.9 × max(63, 81, 100) = **90**.

Example 4: Discounted return. Rewards 0, 0, 100 then stop, γ = 0.9: V = 0 + 0.9 × 0 + 0.81 × 100 = 81.

Key points to remember

- GA: population, fitness, selection, crossover, mutation; roulette Pr = f_i / Σ f.
- GP evolves program trees; Baldwin effect vs Lamarckian evolution.
- Sequential covering learns one rule at a time; FOIL learns first-order Horn clauses with FOIL_Gain.
- Q-learning: Q(s, a) ← r + γ max Q(s', a'); exploration vs exploitation.

⭐ Important questions

1. Explain genetic algorithms: representation, operators, fitness and selection, with an example.
2. Explain hypothesis space search by GA, genetic programming and models of evolution and learning (Baldwin effect).
3. Explain the sequential covering algorithm and Learn-One-Rule.
4. Explain learning sets of first-order rules: FOIL.
5. Explain reinforcement learning and the Q-learning algorithm with an example.

Last-hour checklist

☐ Mitchell's definition (T, P, E); designing the checkers learner; issues.

☐ Find-S and candidate elimination on EnjoySport; version space; inductive bias; 96 instances, 973 hypotheses.

☐ Entropy 0.940; gains 0.247, 0.152, 0.048, 0.029; tree Outlook → Humidity / Wind.

☐ Overfitting, pruning, gain ratio, continuous attributes.

☐ Perceptron for OR (w = (1,1), b = 0); delta rule; Adaline; backprop δ formulas.

☐ Confidence interval (0.158 to 0.442); comparing algorithms with k-fold and paired t-test.

☐ Bayes theorem; MAP and ML; Bayes optimal; Naive Bayes (No, 0.795); belief networks; EM.

☐ PAC; $m \geq (1/\varepsilon)(\ln|H| + \ln(1/\delta))$ (140 examples); VC dimension; mistake bounds.

☐ k-NN example; distance weighting; locally weighted regression; RBF; CBR; lazy vs eager.

☐ GA steps, roulette, crossover, mutation; GP; Baldwin; sequential covering; FOIL_Gain; Q-learning (90).

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Machine Learning	Tom M. Mitchell	McGraw-Hill Education (India) (2013)	WorldCat
2	Introduction to Machine Learning	Ethem Alpaydin	MIT Press	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
3	Machine Learning: An Algorithmic Perspective	Stephen Marsland	CRC Press (2009)	WorldCat
4	Pattern Recognition and Machine Learning	Christopher M. Bishop	Springer	WorldCat

Bishop's "Pattern Recognition and Machine Learning" is available as a free PDF from Microsoft Research with the author's permission.

Syllabus source: AKTU Open Elective syllabus for B.Tech final year, 2025-26 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER
sheerat